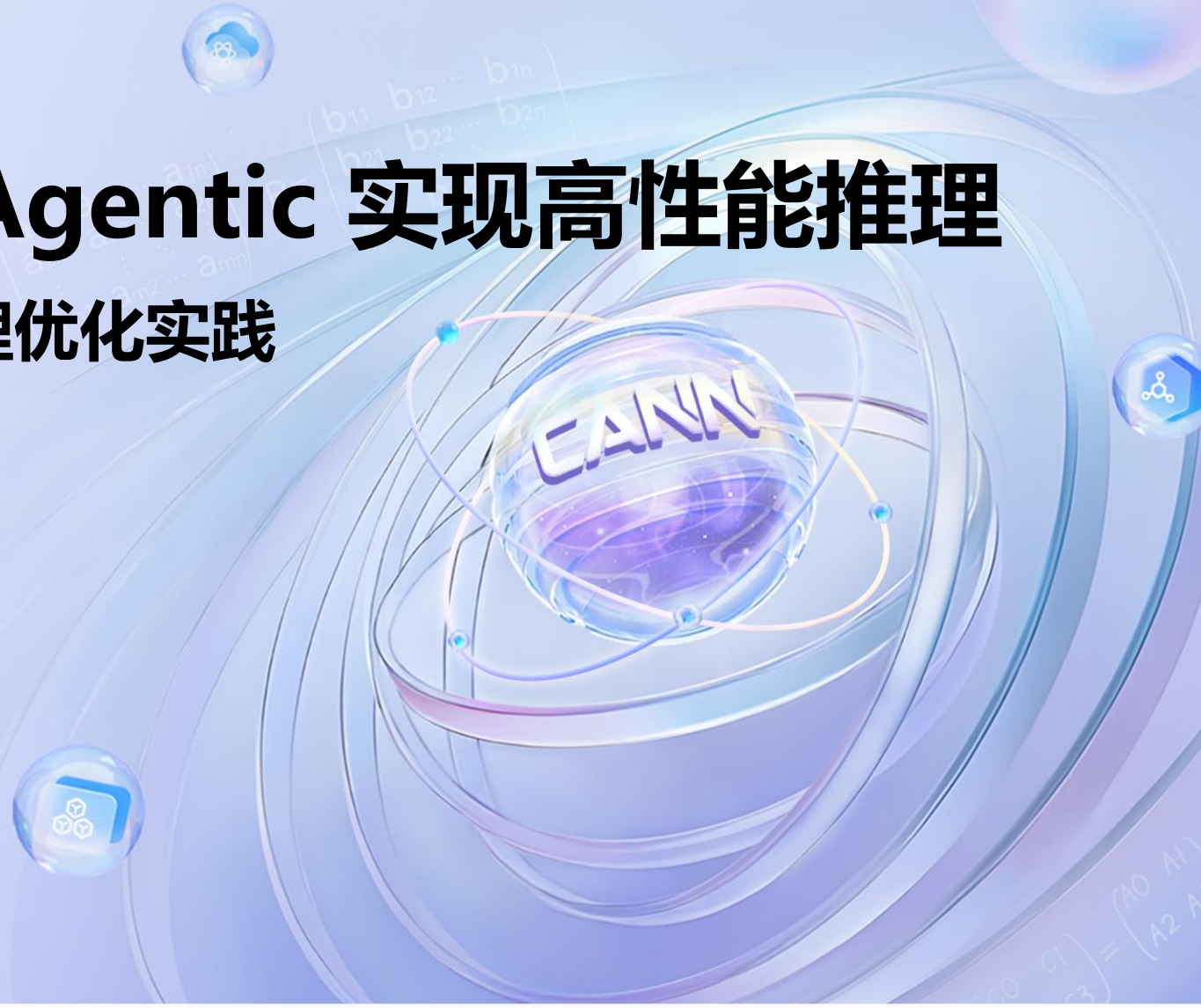
Ascend 950 支撑 Agentic 实现高性能推理

DeepSeek-V4.1-Flash 推理优化实践



CANN

目录

Part 1 模型解析

Part 2 基于 950DT 的整网优化方案解析

Part 3 Future Plan

DeepSeek 架构演进

DeepSeek-V3.2

模型规格

- 总参数 671B / 激活约 37B

注意力机制

- 引入 **DSA 稀疏注意力**，解决长上下文 Attention 计算量过高问题

MoE 结构

- 每层 1 共享 + 256 路由专家
- 每 Token 激活 8 个路由专家

关键词：稀疏注意力

DeepSeek-V4 Pro/Flash

模型规格

- V4-Pro：约 1.6T 总参 / 49B 激活
- V4-Flash：284B 总参 / 13B 激活
- **1M Token** 上下文

混合Attention结构

- Window 128 滑窗注意力
- CSA + HCA 高倍率压缩
- Lightning Indexer 稀疏Attention计算

关键词：百万上下文，Attention 成本进一步降低

DeepSeek-V4.1-Flash

模型规格

- **552B 总参数量 + CED 架构**
- Prefill 激活 8B, Decode 激活 16B
- 1 共享 + 384 路由专家，topK-6

混合Attention + 跨层复用机制

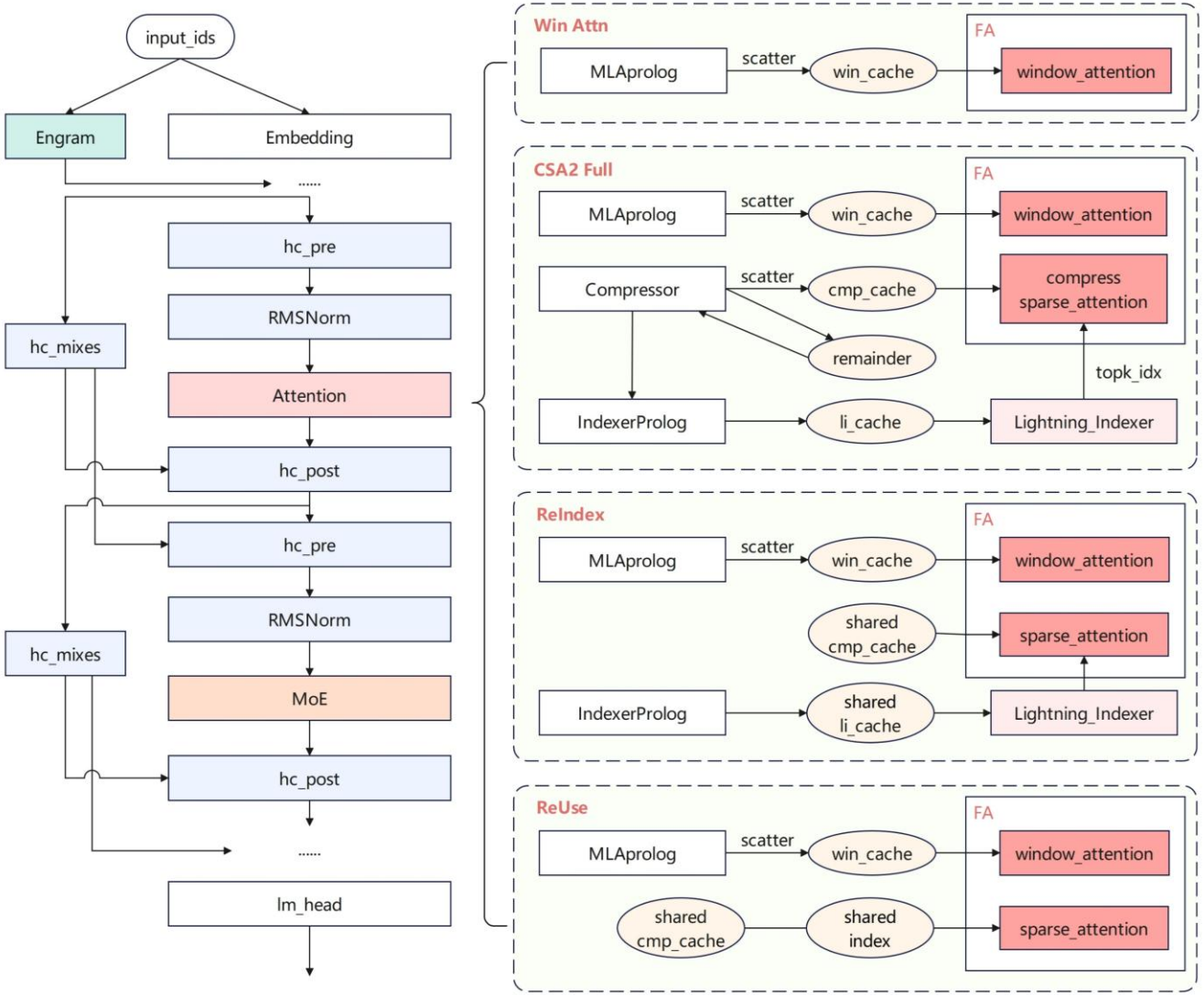
- **CSA2 + SharedKV + FP4 量化**
- 全局 KV Cache 约 890 B/token，约为 V4-Flash 的 **¼**
- **Bounded Replay**：持久化 KV 再降至约 **1/8**

能力扩展

- **原生多模态支持，百万上下文**
- Engram 扩展
- Dspark 替代 MTP

模型解析 —— 整体结构

- **原生多模态支持**：视觉 token 形式与文本 token 在同一 LLM 主干中共同参与推理。
- **Engram 扩展信息**：Engram 通过外置 196B 知识表，并稀疏查表调用，用更小的计算代价获取更多的额外知识。增加模型参数量。
- **CED 架构**： 40 层主干采用 CED 架构（20 层编码器 + 20 层解码器），降低 Prefill 激活参数量。
- **LLM 高倍率稀疏 Attention**：通过 SWA 128窗口大小 + DSA top 512 稀疏 attention，大幅降低 attn 计算量。
- **LLM 跨层 KV Cache 复用**：KV 用 CSA2 压缩，跨层复用 KV 与 Top-K 索引，全局 KV 压缩至 890 B/token。
- **稀疏 MoE**：384 个路由专家，稀疏激活 6 个路由。1 个共享专家。
- **Dspark 投机推理**：3 层 SWA，并行投机 5 个 token，提升投机效果。



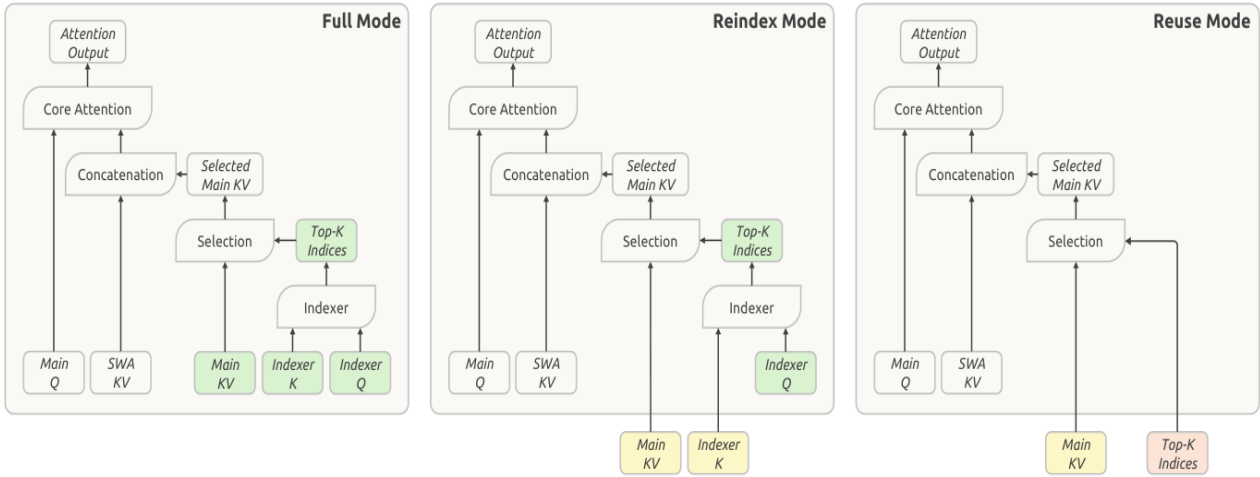
模型解析 —— CSA2

Attention 架构

➢ **多种 Attention 加速长序列计算**

沿用了 SWA + Compress Sparse Attn 的混合注意力机制，对比 DeepSeek-V4:

- 压缩倍率由 4 &128 调整为 1 & 2；
- 去除了 HCA，所有层都进行稀疏 Attention 计算；
- 分为3种模式，进行跨层的 KV Cache 与 Index 复用；
- 所有层都需要做 SWA 计算，window 大小 128。



图源：https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf

三种模式的数据来源对比

模式	Main KV	Indexer K	Top-K	层数
Full	本层生成	本层生成	本层打分	4
Reindex	复用	复用	重打分	4
Reuse	复用	复用	复用	30
SWA	本层生成	/	/	40

40 层的模式分配（静态指定）

层区间	注意力 / 压缩率	模式分组
L0–L1 （2层）	纯 SWA	不压缩，滑窗 warm-up
编码器 L2–L19 （18层）	CSA2，压缩 2倍	**3 组 × 6 层** 首层 Full + 5 层 Reuse
解码器 L20–L39 （20层）	CSA2，不压缩	**5 组 × 4 层** 第 1 组 Full + 3 Reuse； 第 2–5 组 Reindex + 3 Reuse
KV Source Layer	4 层生产 KV	KV source layer × 4: [2, 8, 14, 20]； Index source layer × 8： [2,8,14,20,24,28,32,36]

模型解析 —— Indexer 层级稀疏索引

Indexer Candidate 设计

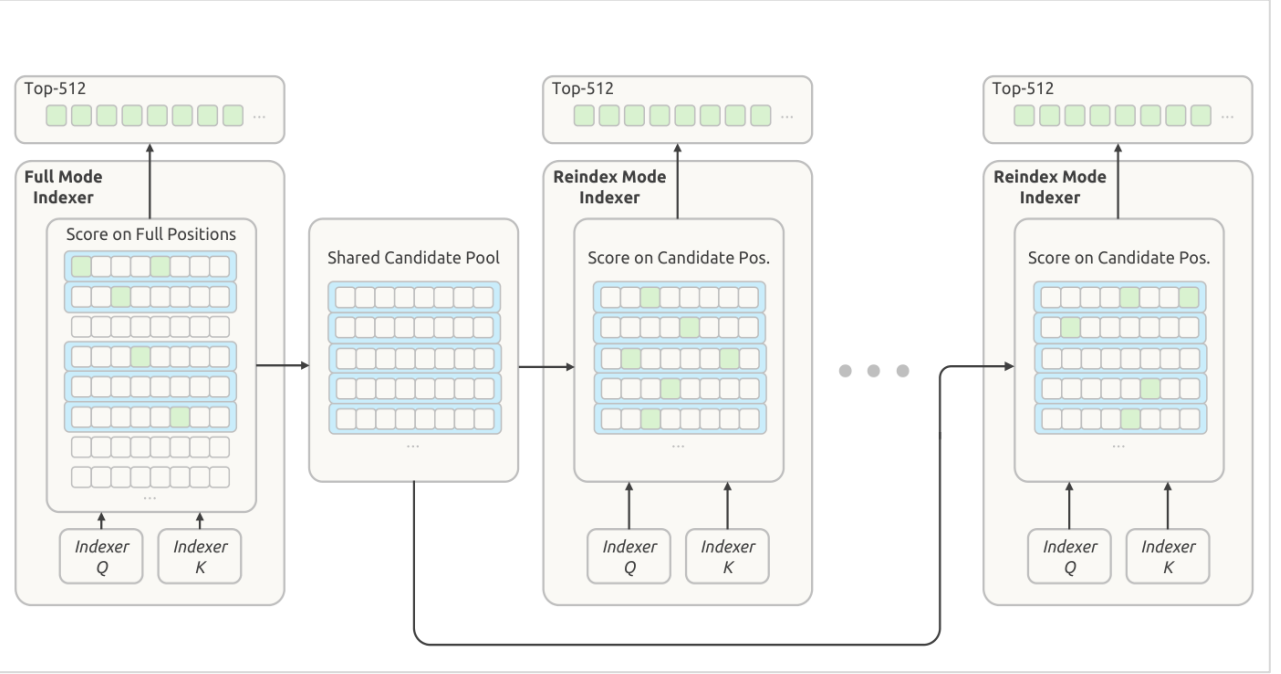
➢ **将随上下文线性增长的 Indexer 计算变为常数级：**

- Indexer 计算量：

$$batch_size * n_heads * head_dim * seq_len * num_bytes$$

- 分层检索：
  1. 在模型的 Layer20 计算候选池：最大 16K
  2. 将 kv 按照 8 个一组分块，每一块选出分数最高的作为代表；
  3. 所有块按代表的分数排序，选最高的 2048 个块，放入候选池；
  4. 后续 reindex 的 4 层只在候选池里打分 & 选 top 512。
- 收益：候选池大小固定后，后续 indexer 每个 query 的打分次数变为常数。

层数	Index K	Top-K	Candidate
L0, L1	/	/	/
L2 ~ L19	Source 层计算 其他复用	Source 层计算 其他复用	/
L20	本层计算	本层计算	本层计算
L21 ~ L39	Source 层计算 其他复用	Source 层计算 其他复用	使用 Candidate

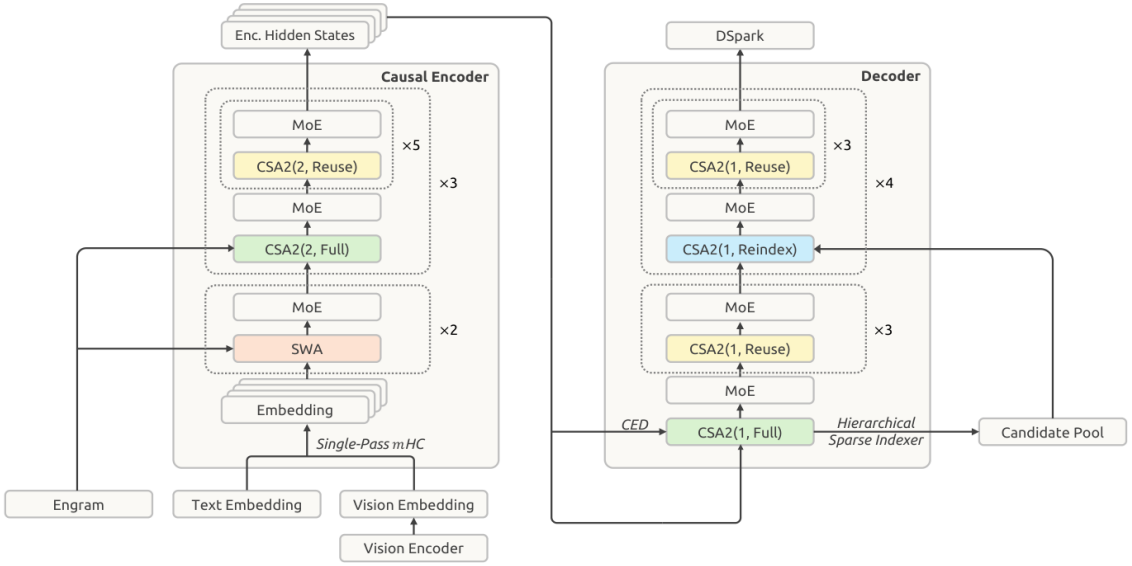


图源：https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf

模型解析 —— Causal Encoder Decoder (CED)

CED 结构

- **20 层编码器 + 20 层解码器**
- 大幅降低 Prefill 计算量；
- Decode 阶段 L21~L39 复用 L20 的 KV Cache & Index Cache;
- Window Cache 逐层保留并传递给 decode 阶段；
- 因此 prefill 在后 19 层仅需要 128 token 参与计算生成；
- Prefill 激活参数量近似减半。



关键模型结构配置

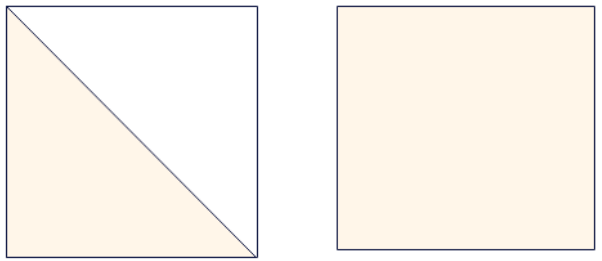
配置项	数值
Transformer 层数	40（20 Enc + 20 Dec）
Hidden size	5,120
Query 头 / 头维度	64 × 512（压缩维 1,280）
Indexer 头 / 头维度	32 × 128
SWA 滑窗 n_win	128（每层保留）
Compress KV Cache	**保留 4 层，其余共享**
Index KV Cache	**保留 4 层，其余共享**
Index Top-K	**512， 计算 8 层，其余共享**
MoE 专家	1 共享 + 384 路由，激活 6 个
专家中间维度	2,304（SwiGLU，clamp=10）
mHC 残差流	4 路（Single-Pass）
Engram 参数	160000 × 3 × 256 × 8

图源：https://huggingface.co/deepseek-ai/DeepSeek-V4.1-Flash/resolve/main/DeepSeek_V41_Tech_Report.pdf

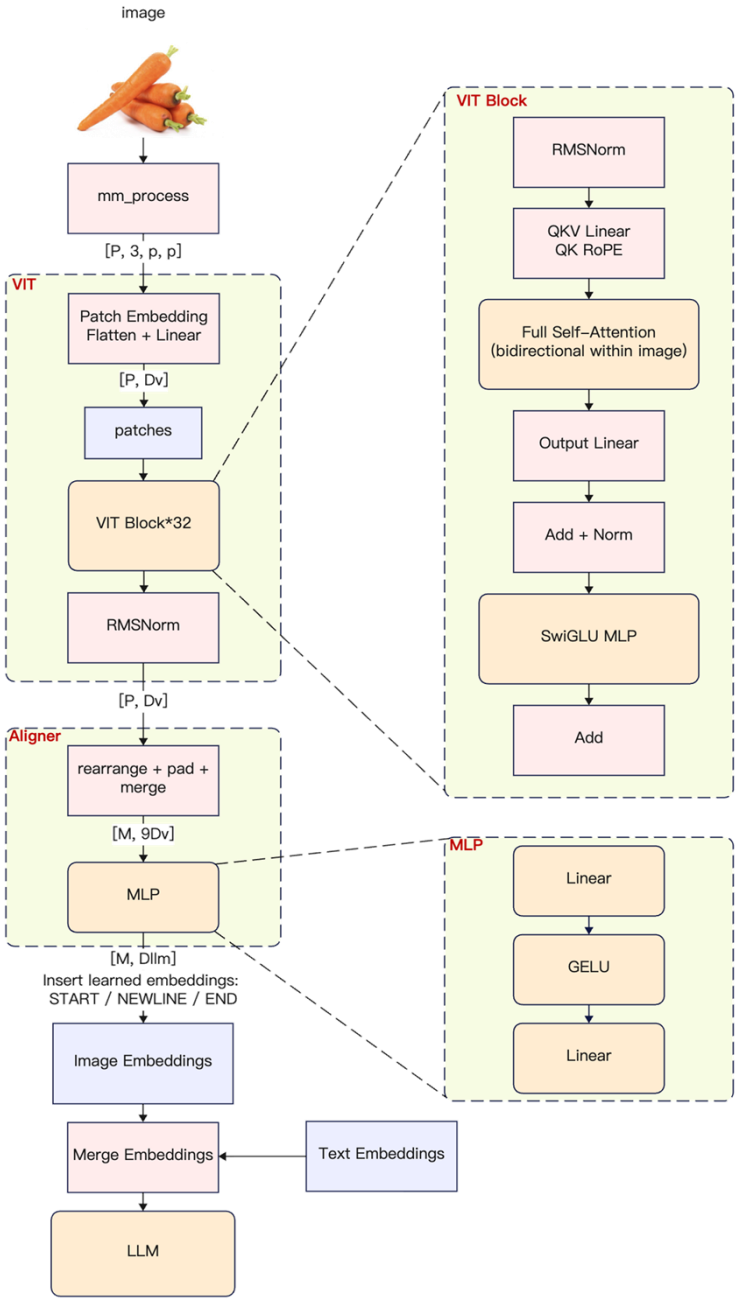
模型解析 —— ViT 多模态原生支持

整体架构

- **ViT结构**：ViT block 为 32 层 MHA+MLP、隐藏维度 1024，patch size 14。
- **输入处理**：任意长宽比的图像直接输入，大图等比缩放；ViT 输出时，经过 3×3 pixel-unshuffle 把视觉 token 数降 9 倍，单图最多 1024 个视觉 token。
- **LLM衔接**：视觉 token 经过 2 层 MLP Aligner 投影到 hidden_size 5120，插入输入序列中图像所在的位置，与文本 token 一同进 LLM 主干；图像内部按行展平、逐行插入换行分隔符以保留 2D 空间语义。
- **注意力**：ViT 内部为双向注意力，LLM 主干为因果注意力。
- **Engram**：图像 token 在 Engram 记忆中被跳过，n-gram 不统计图像 token。



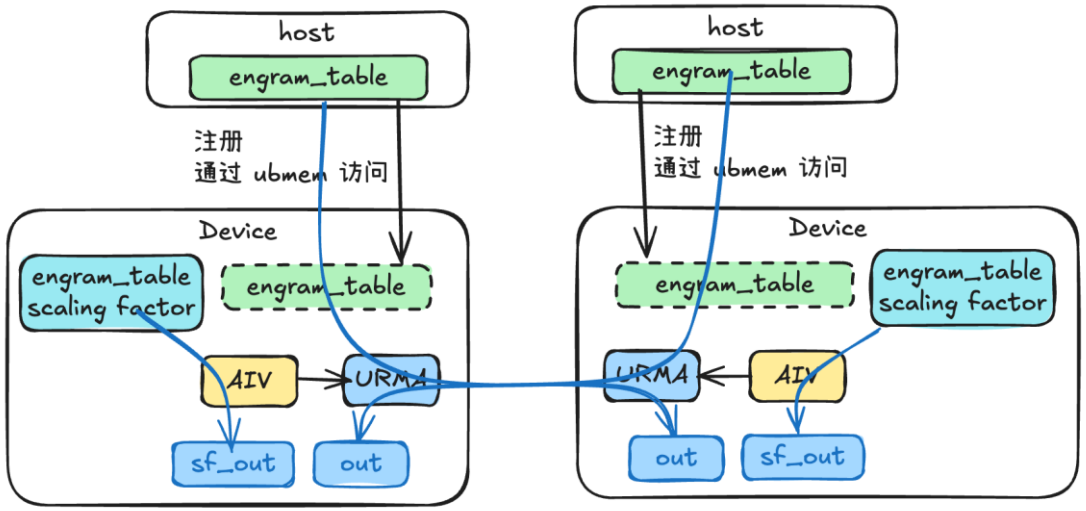
causal 因果注意力　　ViT 全注意力

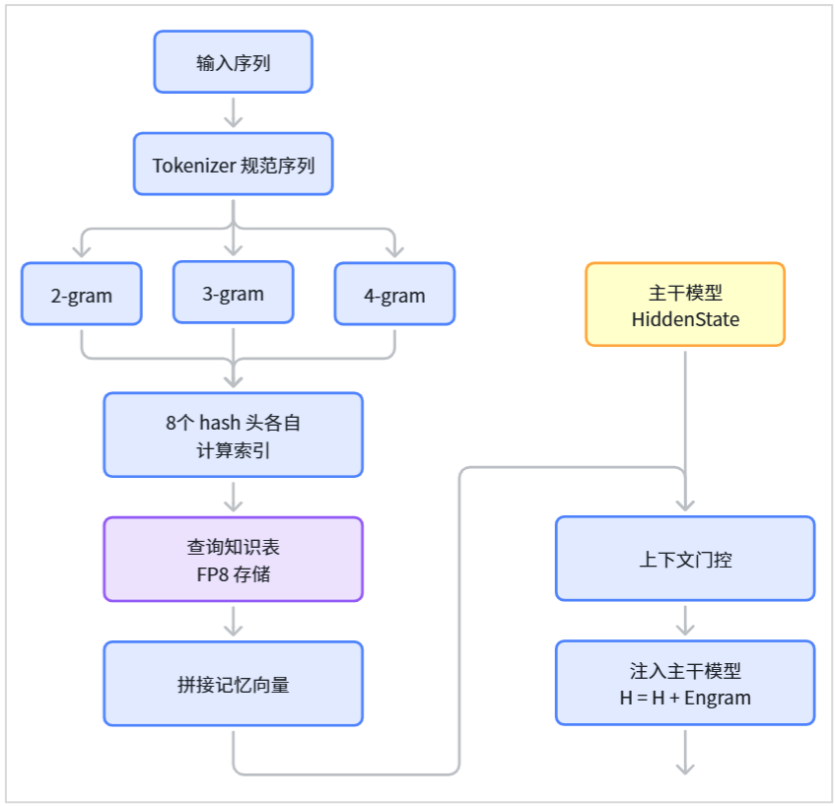


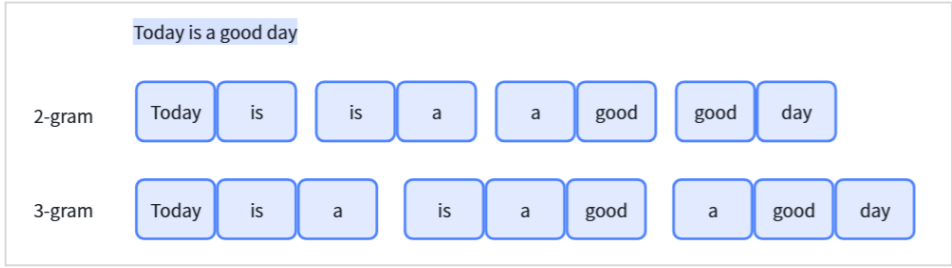
模型解析 —— Engram

整体架构

- **Engram结构**：外置知识表，3 阶 n-gram，每阶 8 个 hash 头，每张表 16M 条目，总参数量 196 B。穿插在模型的 1 & 14 层。
- **Engram 条件记忆**：通过稀疏查表记忆换取额外知识，几乎不增推理计算开销。
- **量化**：FP8 存储知识表，占用约 200G 内存。
- **Offload 方案**：engram 表占用内存较大，通过 offload 放到 host 侧，减少 device 内存占用。充分利用 UB 能力，不占用计算核，可与计算 overlap。







模型解析 —— Single-Pass mHC

mHC 结构

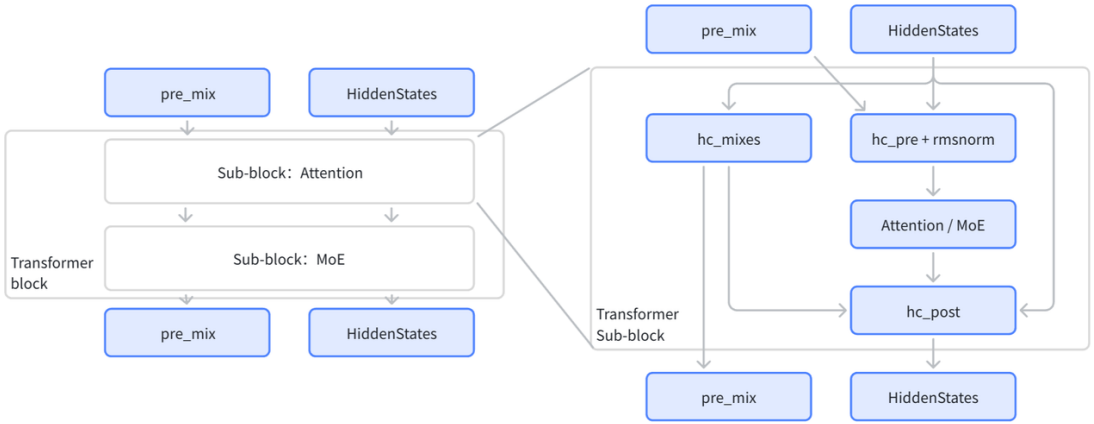
mHC 的实现包含三个步骤：「残差更新 + 输入混合 + 系数预测」

➢ **Multi-Pass mHC (DeepSeek-V4)**

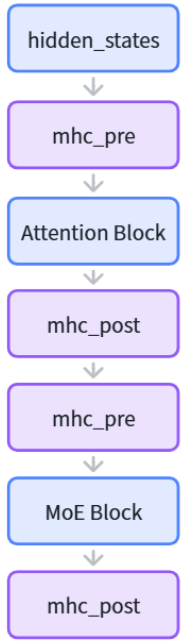
- 串行执行上述三步，步骤间紧密的数据依赖；
- Kernel 需要对残差流多次连续读写；
- 导致激活内存流量是理想下界的 2 倍。

➢ **Single-Pass mHC （DeepSeek-V4.1-Flash）**

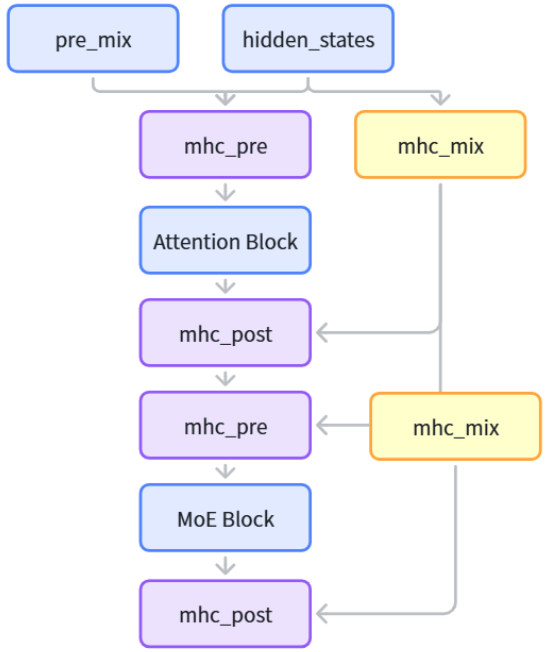
- 把输入混合系数错位一个 block，消除了系数预测与输入混合之间的数据依赖；
- 三个步骤融合为单 kernel 的 Mega-mHC，把激活内存流量再减半。



DS-V4.1 single pass mHC 示意图



DSV4



DSV4.1

模型解析 —— 计算量与内存分析

模型计算量

➢ Attention 计算量

Full Attention：

- 计算复杂度为 $O(L^2 d)$，L 为 Query 序列长度；

CSA的整体计算量下降显著：

- **SWA**：计算复杂度$O(L \cdot \text{win_size} \cdot \text{d})$；全部40层都有；
- **CSA2 ratio=1**：L20~39；稀疏topk 512，attention $O(Lkd)$，k=512；Indexer 仅在 L20 从全量序列中选择 TopK，4 个 reindex 层从侯选池中选择 TopK，计算复杂度控制在 $O((\text{pool_size})^2)$，并共享给后继层。
- **CSA2 ratio=2**：L2~19；稀疏 topk 512，attention $O(Lkd)$，k=512；Indexer仅在 3 个 source 层计算 TopK，共享给后继层；Indexer 对压缩 2 倍后的 KV 计算，计算复杂度减少 4 倍 $O((\frac{L}{2})^2)$；
- 增加了 Compressor 压缩 KV Cache 的计算开销。

➢ MoE计算量

在 384 个路由专家中，单 token 激活 6 个路由专家和 1 个共享专家，计算参数量为：

$$(top_6 + 1) * hidden_size * \text{intermediate_size} * 3$$

内存分析

➢ 权重内存

整体参数 552B：

- MoE 专家个体减小，在 MXFP4/MXFP8 量化下，专家权重约 270GB；
- Attention 模块权重约 4.9GB，包括 Compressor 及 Indexer；
- Engram 模块约 189GB；
- Vision 模块包括 ViT+Aligner，权重约 0.9GB。

➢ Cache内存

- Window Cache：window size 128 长度的cache空间：

$$\text{batch_size} * \text{win_size} * \text{head_dim} * \text{storage_bytes} * \text{num_layers}$$

- Compressed Cache 压缩后的 KV Cache，仅 L2、8、14、20 持有，后继层复用：

$$\text{batch_size} * \frac{\text{kv_length}}{compress_ratio} * \text{head_dim} * \text{storage_bytes} * \text{num_source_layers}$$

- Remainder Cache 不满足压缩长度的状态特征，暂存在 Cache 内，仅 L2、8、14持有：

$$\text{batch_size} * (ratio - 1) * \text{comp_dim} * \text{storage_bytes} * \text{num_remainder_layers}$$

目录

Part 1 模型解析

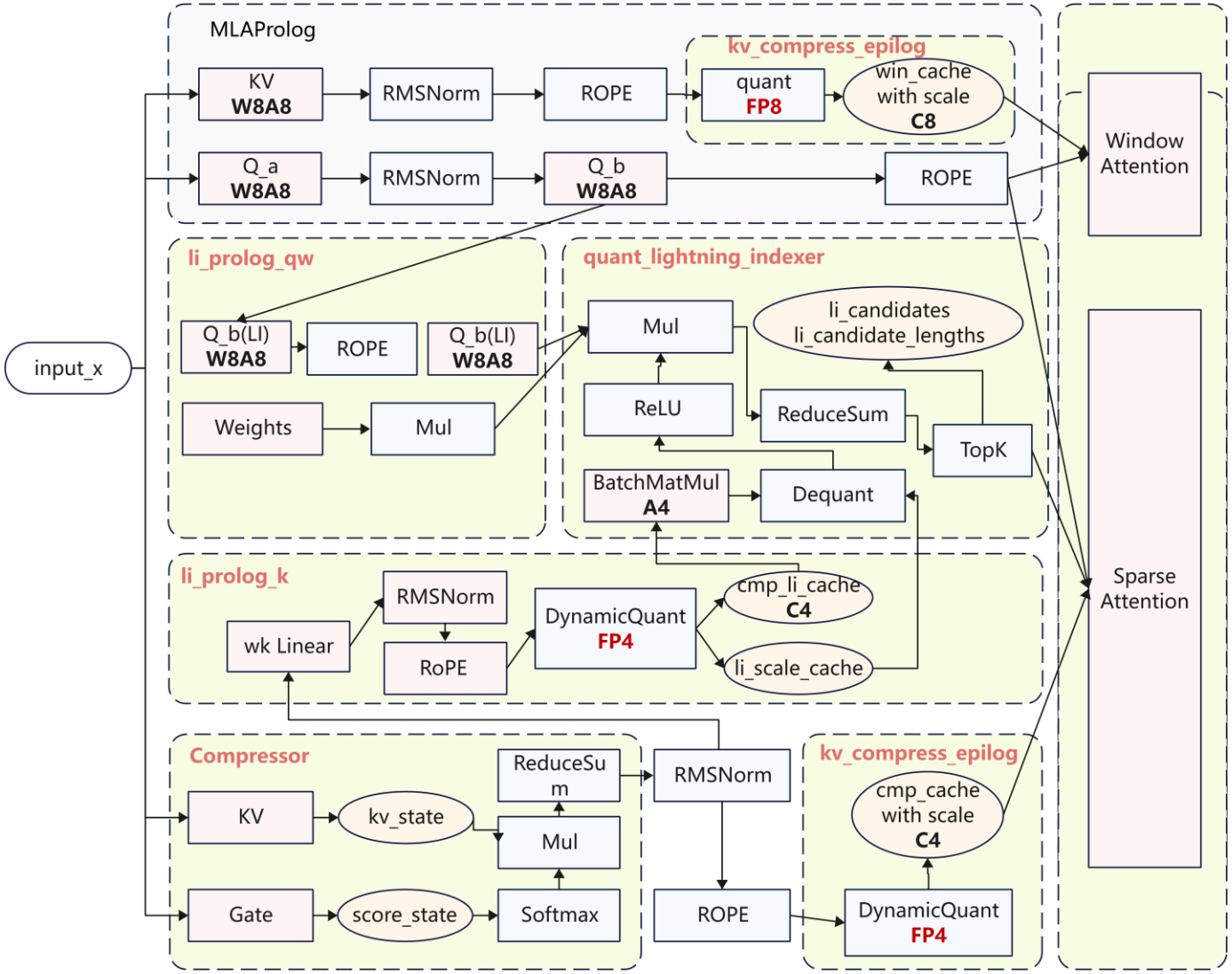
Part 2 基于 950DT 的整网优化方案解析

Part 3 Future Plan

整网优化方案解析 —— 量化策略

950 DT

- **权重与激活**
  1. 共享专家采用 MXFP8 A8W8 全量化计算；
  2. MoE 专家采用 A8W4 伪量化计算，权重使用 MXFP4 存储；
  3. Matmul 基本采用 MXFP8 A8W8 全量化计算。
- **KV Cache 分精度存储**
  1. 滑窗 KV 用 FP8 E4M3（per 32 elements)；
  2. 压缩 KV 用 FP4（per 16 elements）；
  3. Attention 采用混合 A16-C8-C4 伪量化计算
  4. IndexCache 用 FP4（per 32 elements）；
  5. Lightning Indexer 采用 A4C4 全量化计算；
  6. Compress State 使用 FP32 存储。
- **Engram 表**
  1. 表采用 FP8 存储，查询时按行反量化到 BF16。

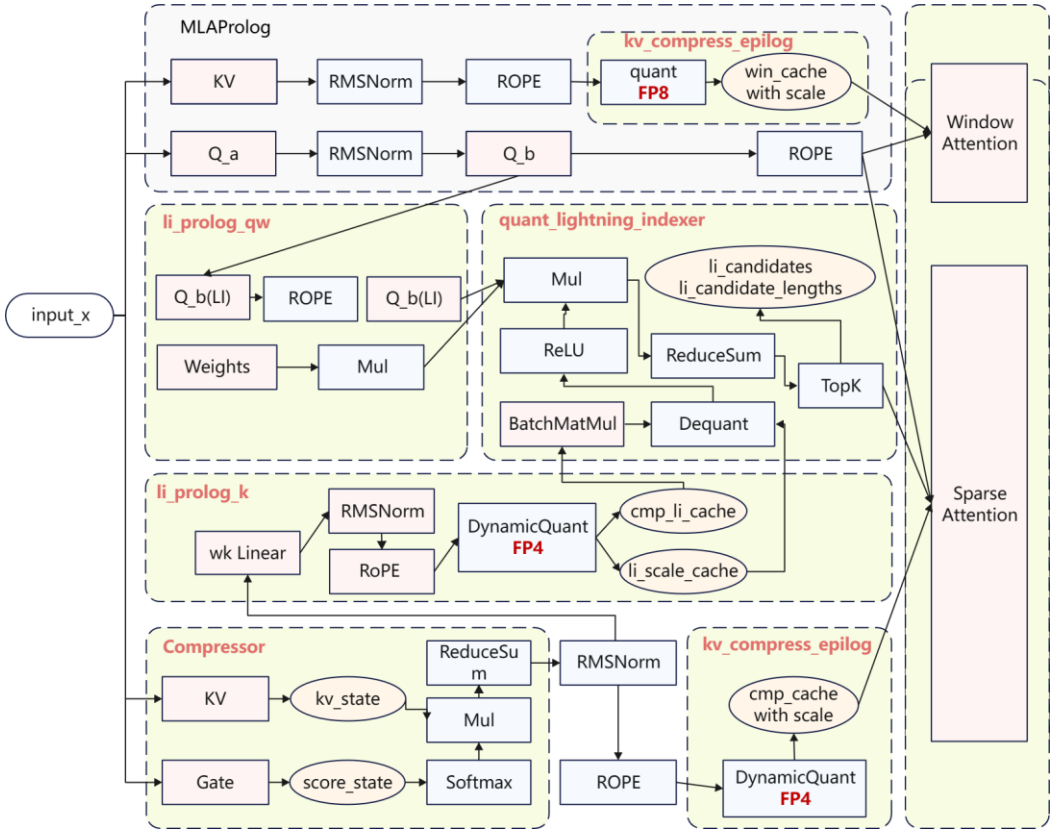


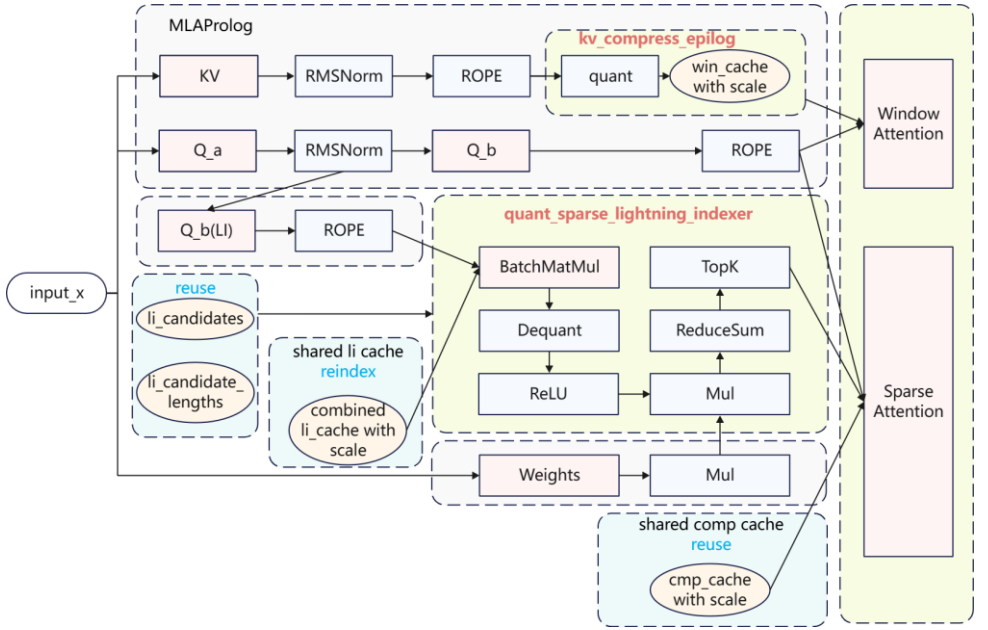
整网优化方案解析 —— 950 原生融合Kernel

Attention

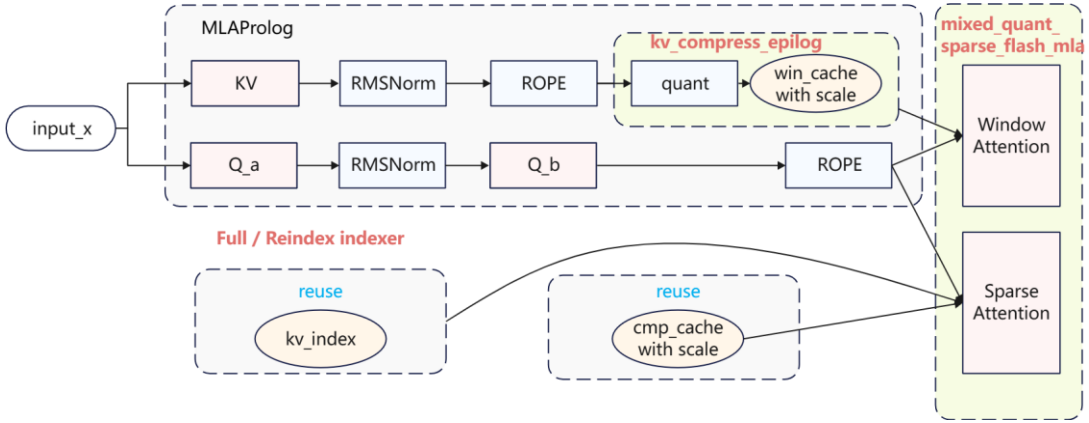
➢ **针对Attention模块，开源提供高性能融合算子：**

- MLAProlog：融合 Query、KV 投影与滑窗 KV Cache 写入；
- IndexerProlog + LightningIndexer：融合 indexer 的 Q/K 投影、量化与候选检索；
- Compressor 与 KV Cache 写回：融合压缩计算与量化写回；
- SparseAttention：融合 SWA 与压缩稀疏两路 KV 的混合注意力。





Reindex Layer

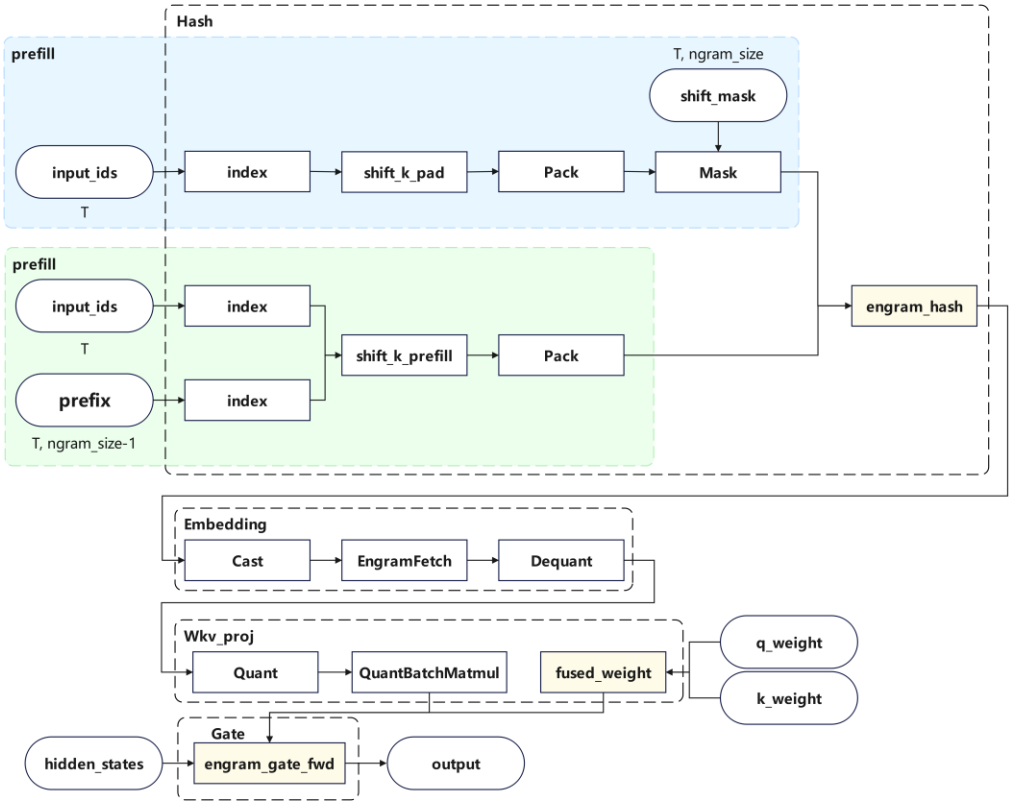


Reuse Layer

整网优化方案解析 —— DeepSeek 官方开源 Kernel

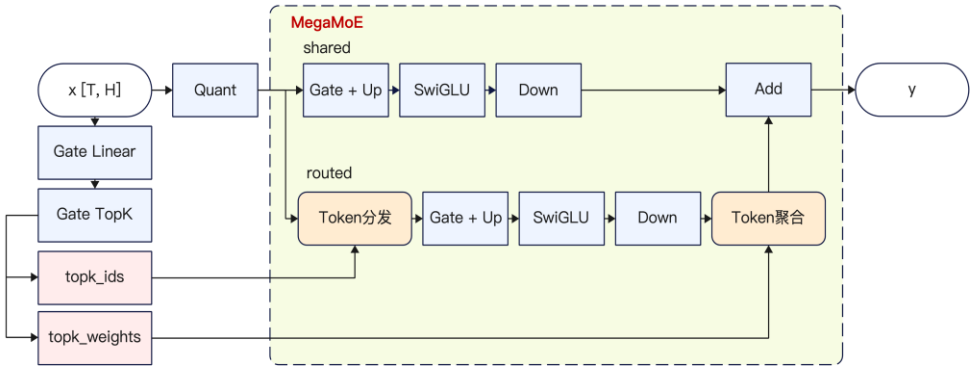
TileKernels

- Prefill 使用基于 TileLang 支持的DS原生开源Kernel：
  - MoEGateTopK
  - mHC
  - engram_hash
  - fused_weight
  - engram_gate_fwd



DeepGemm MegaMoE

- 将token分发、两次专家 Linear、SwiGLU 和 token 聚合融合，并在算子内部完成 Shared Expert 计算及结果相加，通过通信与计算的流水重叠减少阶段间等待。



Tile Kernels

TileKernels is a library of dozens of highly optimized kernels implemented in TileLang, a domain-specific language supporting multiple hardware backends. It provides kernels for several common operations in LLM training and inference, including mixture-of-experts routing, Engram, quantization, and manifold hyper-connections. Most kernels achieve performance close to the hardware's compute or memory bandwidth limits. All of these kernels have already been used in our internal training and inference workloads.

> TileKernels 是一个包含数十个深度优化算子的高性能算子库，基于支持多种硬件后端的领域专用语言 TileLang 实现。它为大语言模型训练与推理中的几项常见操作提供算子，包括混合专家路由、Engram、量化和流形超连接（mHC）。大多数算子的性能接近硬件的计算吞吐或内存带宽上限。全部算子已用于我们的内部训练与推理任务。

News

- **[2026-09-30] Huawei Ascend support**: Added Huawei Ascend support and updated the usage documentation. Following the NVIDIA path, the kernels now ship a second backend that is selected automatically at runtime, so the same Python APIs run on both NVIDIA GPUs and Huawei NPUs.

整网优化方案解析 —— 并行策略

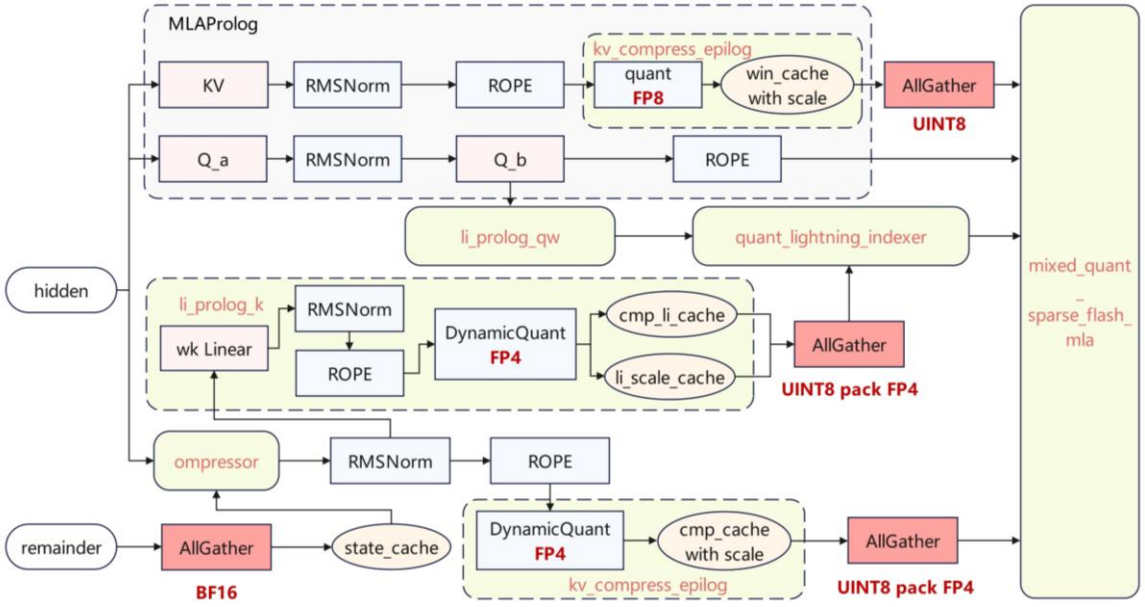
Prefill Context Parallel & Expert Parallel

➢ 长序列场景使用CP并行，优化TTFT：

- 对 Prefill 输入沿序列维度切分于不同卡上，并通过 zig-zag 方式使卡间计算量相对均衡；
- 通过 AllGather 获取各个卡需要的数据依赖：
  1. 前序 chunk 的 remainder 和 win kv；
  2. 完整的 KV Cache 和 Index KV;
  3. 通信量化后的 KV。



ZigZag 部署

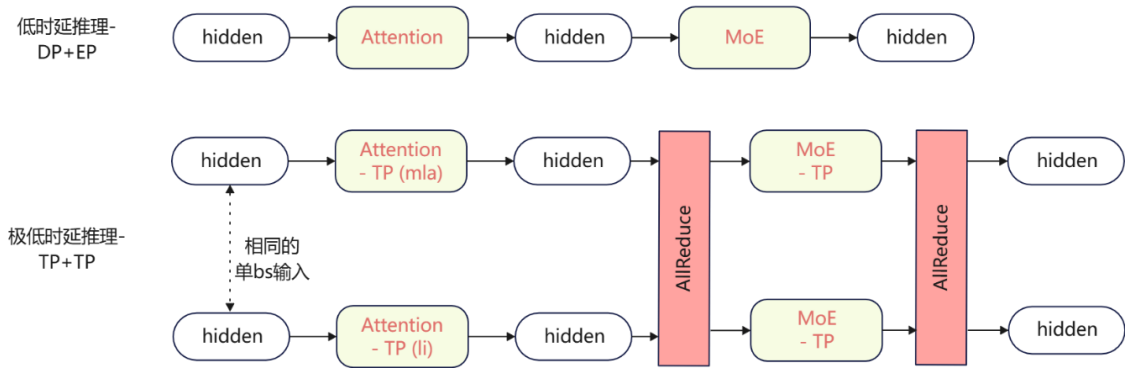


CP 数据传递路径

Decode Data Parallel & Expert Parallel

➢ Decode大EP部署实现低时延高吞吐

- Attention 采用 DP 并行部署减少通信开销；
- MoE 采用 EP 并行部署降低单卡计算量；
- LM_Head 使用局部 TP 切分；
- Engram 支持多卡 TP 切分，并支持 offload 到 host 侧，降低 device 内存占用；
- 极低时延场景可进一步对 Attention / MOE 做 TP 切分。



Decode 低时延/极低时延并行部署

整网优化方案解析 —— 多流并行

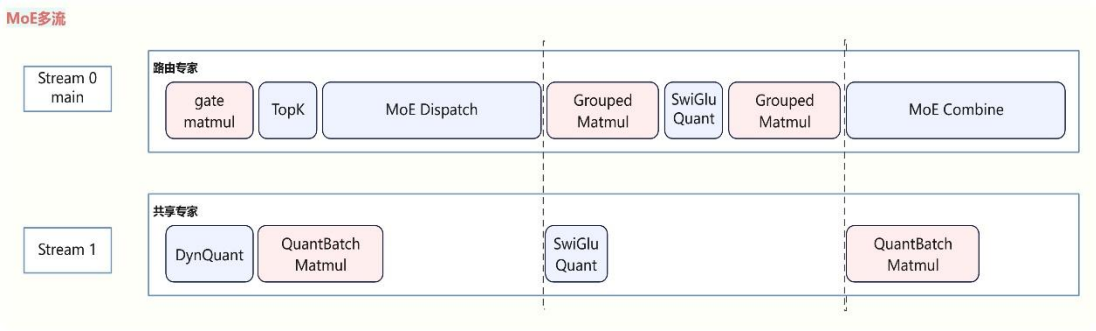
多流并行

➢ **灵活并行最大化利用算力**

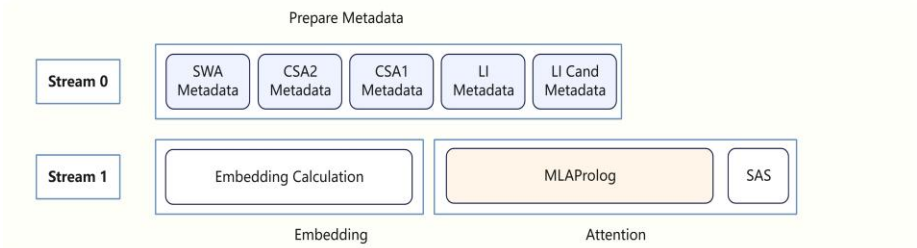
- 没有数据依赖的计算、通信算子，可以在多条计算流上并行执行；
- 通过合理设置多流并行的位置，可以最大化利用计算资源。

MoE专家多流 & AICPU 多流

➢ **共享专家&路由专家并行**

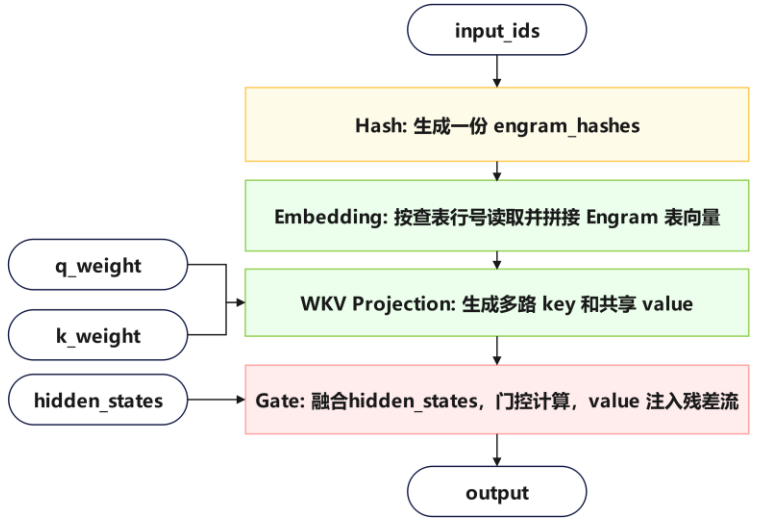


➢ **AICPU Metadata 算子并行**

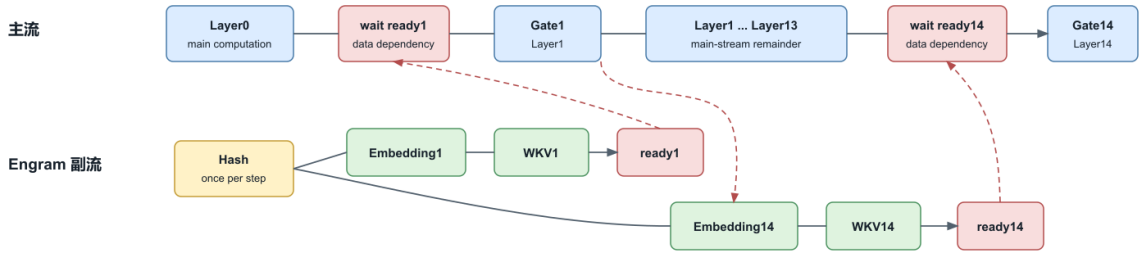


Engram 多流

从调度角度，Engram 前向过程可以拆分为四个阶段：



- Hash、Embedding 和 WKV Projection 不依赖 hidden_states，可以并行计算；
- Gate 依赖 engram 结果与 hidden_states，是多流计算的汇合点。

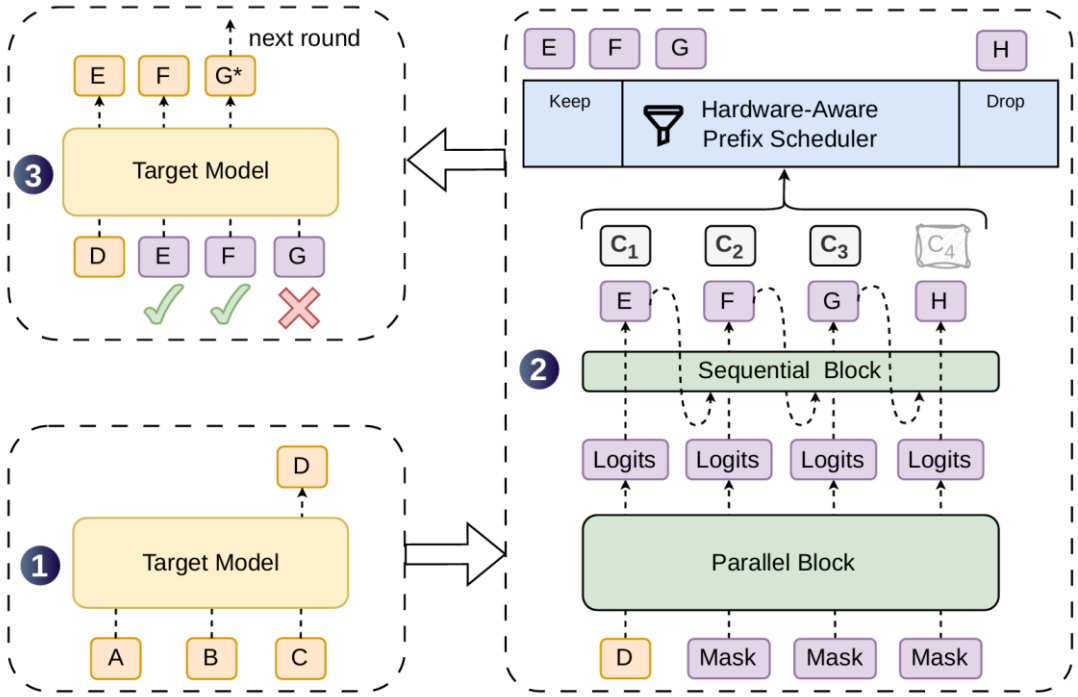


整网优化方案解析 —— DSpark

DSpark 投机推理

➢ Dspark投机推理

- 在一次草稿模型执行中生成一组 Draft Token，再由主模型批量 Verify。
- DSpark Spec model 收集 Main model [37, 38, 39] 层的 hidden states，用来构建历史上下文。

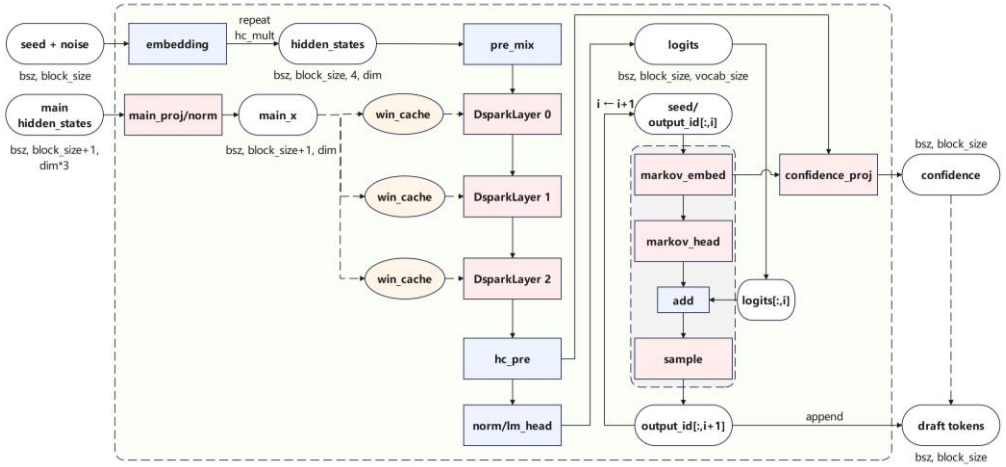


图源：DeepSeek-AI 官方论文《DSpark: Confidence-Scheduled Speculative Decoding with Semi-Autoregressive Generation》，https://arxiv.org/pdf/2607.05147

DSpark 模型

➢ DSpark Spec model

- **模型结构**：DSpark 使用多级 Proposal Layer 生成 Draft Block，通过 Markov Head 引入块内 Token 的条件依赖、串行生成 Draft Token，并由 Confidence Head 给出候选置信度。



- **Cache 结构**：在 DSpark 实现中，每层 Proposal Layer 仅使用 window attention，所以每层只拥有一份KV Cache。
- **MoE 结构**：

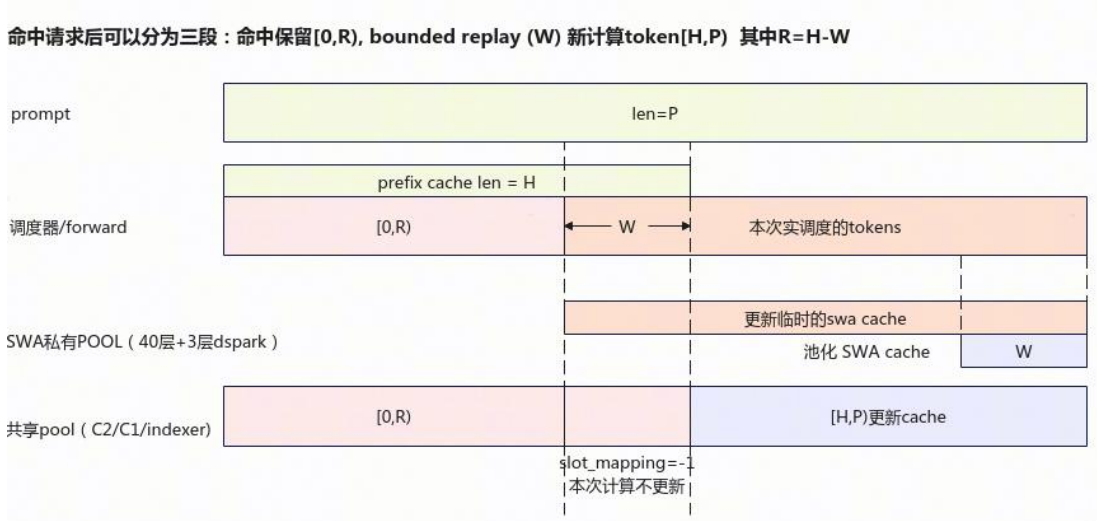
Model	路由专家	激活专家
Main model	384	6
Dspark model	128	3

整网优化方案解析 —— 框架部署

SWA Bounded Replay

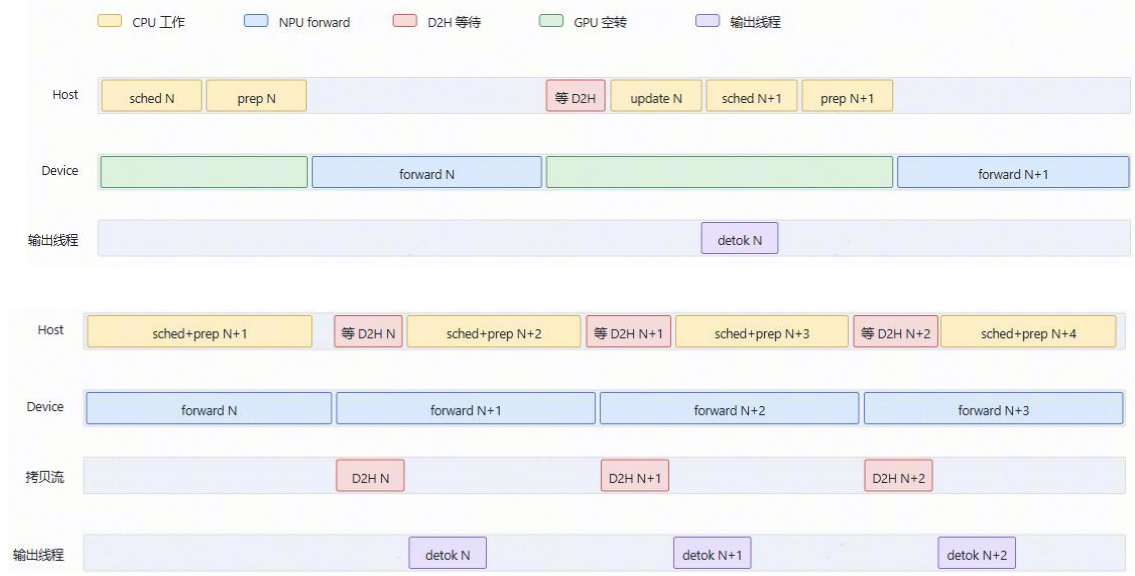
- **背景：**在框架部署的 prefix cache 场景中，SWA KV的存储占比很大；为了降低部署成本，可以使用 SWA Bounded Replay。
- **方案：**不缓存 SWA KV，只缓存 Main KV 和 Indexer KV，并额外重算命中前缀末尾的 win_size 个 token，重建 SWA KV。

框架中 SWA 使用私有 Pool 进行 cache 管理，大小只和 win size 以及请求个数相关。



异步调度

- **背景：**vllm 默认的执行流程中，Device 上每个step都有迭代间隙。NPU 上 ACL Graph 把 forward 压缩得极短之后，这段 CPU 时间占比成为主要瓶颈。
- **方案：**维护一个深度为 2 的 in-flight batch 队列——当前步在 device 上跑的同时，CPU 并行地为下一步做调度和输入准备， 把 CPU 的工作全部“藏”进 forward 的执行时间里。



昇腾 950DT 性能 Benchmark

- **低时延场景 TPOT 小于 5ms 并实现 2727 吞吐，高并发吞吐可达 5800 TPS@13.79ms。**

在 Ascend 950DT 平台上，使用 32 卡部署 FP8-FP4 混合精度模型，部署策略采用 Attention Data Parallel (DP) 和 MoE Expert Parallel (EP) 并行，在低时延和高吞吐场景均能发挥性能优势。

Global Batch Size	Chips	Dspark	Seq Length	TPOT (ms)	Throughput (Tokens/p/s)
384	32	5	131072	4.40	2727
1536	32	5	131072	9.41	5102
2560	32	5	131072	13.79	5800

注：性能数据基于 Dspark 投机推理与强制 EPLB 配置采集，Dspark 5 场景下平均接受 token 为 4.1个，用户可按照数据集的实际接受率自行折算 benchmark 性能。

整网优化总结

部署策略

- Prefill CP+EP；
- Decode DP+EP;
- Embedding/LM Head TP;
- Engram TP

低比特量化

Weight W8A8 FP8/MXFP8；MoE W4A8 MXFP4 ；KV Cache伪量化 A16-C8-C4 混合精度；

Indexer Cache A4C4 全量化计算;

融合Kernel

- Sparse Attention、Lightning Indexer、MLAProlog、IndexerProlog、MegaMoE
- GroupedMatmul、DequantSwigluQuant;
- DeepSeek 官方开源的 TileKernels，MegaMoE 等高性能算子；

多流并行

MoE 共享/路由专家多流、Engram Offload 多流等;

NpuGraph_Ex

Dspark 投机推理

目录

Part 1 模型解析

Part 2 基于 950DT 的整网优化方案解析

Part 3 Future Plan

Future Plan

- **Tilelang NpuGraph_Ex 支持：**支持 npugraph_ex 入图，进一步扩大应用场景，支持高性能训推；
- **更多融合算子**：attn_prolog, attn_epilog 等融合算子开源支持；
- **Prefill CED 独立部署：**支持 CED 架构下，encoder-decoder 分离部署，提升端到端性价比。

欢迎体验和贡献

cann-recipes-infer

cann-recipes-train

cann-recipes-infer:
https://gitcode.com/cann/cann-recipes-infer

cann-recipes-train:
https://gitcode.com/cann/cann-recipes-train

cann-recipes-sig小组

cann-recipes交流群

欢迎广大开发者体验并参与贡献，如有疑问也可通过issue、SIG或者cann-recipes交流群联系我们！

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同**共享**共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann